SIEMENS

Functions for accessing devices, networks and connections 1

Functions on networks 2

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

12/2025

NOTICE

This document was downloaded from the following site:

https://docs.industrial-operations-x.siemens.cloud

Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided.

Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

⚠ WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

12/2025 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1 Functions for accessing devices, networks and connections ... 7

1.1 Open the 'Devices & networks' editor ... 7
1.2 Querying PLC and HMI targets ... 8
1.3 Accessing attributes of an address object ... 10
1.4 Accessing the channels of a module ... 13

2 Functions on networks ... 15

2.1 Creating a subnet ... 15
2.2 Accessing subnets ... 16
2.3 Accessing internal subnets ... 18
2.4 Get type identifier of subnets ... 18
2.5 Accessing attributes of a subnet ... 20
2.6 Deleting a global subnet ... 31
2.7 Enumerate all participants of a subnet ... 32
2.8 Enumerate IO systems of a subnet ... 32
2.9 Accessing nodes ... 33
2.10 Accessing attributes of a node ... 34
2.11 Connecting a node to a subnet ... 39
2.12 Disconnect a node from a subnet ... 40
2.13 Creating an IO system ... 40
2.14 Accessing the attributes of an IO system ... 41
2.15 Connecting an IO connector to an IO system ... 42
2.16 Get master system or IO system of an interface ... 43
2.17 Get an IO Controller ... 44
2.18 Get an IO Connector ... 45
2.19 Disconnecting an IO connector from an IO system or a DP mastersystem ... 46
2.20 Accessing attributes of a DP mastersystem ... 47
2.21 Accessing attributes of PN Driver ... 48
2.22 Accessing attributes of a profinet io system ... 49
2.23 Deleting a DP master system ... 51
2.24 Deleting a profinet io system ... 51
2.25 Creating a DP master system ... 52
2.26 Accessing port interconnection information of port device item ... 53
2.27 Attributes of port inter-connection ... 54
2.28 Accessing the attributes of a port ... 58
2.29 Enumerate DP master systems of a subnet ... 59
2.30 Enumerate assigned IO connectors ... 60
2.31 Connecting a DP IO connector to a DP master system ... 61
2.32 Accessing AS-i profile and parameter attributes for virtual slaves ... 62
2.33 Accessing CM DP as DP slave and transfer area ... 64

2.34 Coupling 1516 isochron central and decentral Profinet 66
2.35 Openness for CP 1604/CP 1616/CP 1626 68
2.36 Openness transfer areas for PnPn coupler 74
2.37 Openness virtual modules/submodules for ET 200SP PN HF 81
2.38 Accessing domain settings 83
2.39 Accessing redundancy mode attribute 87
2.40 Creating CCDX transfer area 88
2.41 Deleting CCDX transfer area 90
2.42 Modifying CCDX transfer areas configuration 91

1 Functions for accessing devices, networks and connections

1.1 Open the 'Devices & networks' editor

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

You can open the "Devices & networks" editor via the API interface by using one of two methods:

- `ShowHwEditor(View.Topology` or`View.Network`or`View.Device)`: Open the "Devices & networks" editor from the project.
- `ShowInEditor(View.Topology` or`View.Network`or`View.Device)` : Displays the specified device in the "Devices & networks" editor.

Use the `View` parameter to define the view that is displayed when you open the editor:

- `View.Topology`
- `View.Network`
- `View.Device`

Program code

Modify the following program code to open the "Devices & networks" editor:

```
Copies the following program code to clipboard.
// Open topology view from project
private static void
OpenEditorDevicesAndNetworksFromProject(Project project)
{
    project.ShowHwEditor(Siemens.Engineering.HW.View.Topology);
}
```

Modify the following program code to open the "Devices & networks" editor for a device:

```
Copies the following program code to clipboard.
// Open topology view for given device
private static void
OpenEditorDevicesAndNetworksFromDevice(Device device)
{
    device.ShowInEditor(Siemens.Engineering.HW.View.Topology);
}
```

See also

Importing configuration data

1.2 Querying PLC and HMI targets

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

You can determine whether a software base can be used as PLC target (PlcSoftware) or HMI target in the TIA Portal Openness API.

Program code: PLC target

Modify the following program code to determine if a device item can be used as PLC target:

```
Copies the following program code to clipboard.
// Returns PlcSoftware
private PlcSoftware GetPlcSoftware(Device device)
{
    DeviceItemComposition deviceItemComposition =
device.DeviceItems;
    foreach (DeviceItem deviceItem in deviceItemComposition)
    {
```

```
        SoftwareContainer softwareContainer =
deviceItem.GetService<SoftwareContainer>();
        if (softwareContainer != null)
        {
            Software softwareBase =
softwareContainer.Software;
            PlcSoftware plcSoftware = softwareBase as
PlcSoftware;
            return plcSoftware;
        }
    }
    return null;
 }
```

Program code: HMI target

Modify the following program code to determine if a device item can be used as HMI target:

```
 Copies the following program code to clipboard.
//Checks whether a device is of type hmitarget
private HmiTarget GetHmiTarget(Device device)
{
    DeviceItemComposition deviceItemComposition =
device.DeviceItems;
    foreach (DeviceItem deviceItem in deviceItemComposition)
    {
        SoftwareContainer softwareContainer =
deviceItem.GetService<SoftwareContainer>();
        if (softwareContainer != null)
        {
            Software softwareBase =
softwareContainer.Software;
            HmiTarget hmiTarget = softwareBase as HmiTarget;
            return hmiTarget;
```

```
        }
    }
    return null;
}
```

See also

Enumerating devices

1.3 Accessing attributes of an address object

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- For writing access, the PLC is offline.

Application

You can use the TIA Portal Openness API interface to get or set attributes of the address object.

Further you can assign the current process image to an OB.

The following attributes can be accessed:

Attribute name	Data type	Writeable	Access	Description
`IsochronousMode`	Bool	r/w	Dynamic attribute	Activate/Deactivate isochronousMode
`ProcessImage`	Int32	r/w	Dynamic attribute	Set/Get process image partition number.
`InterruptObNumber`	Int64	r/w	Dynamic attribute	Set/Get interrupt organization block number. (classic controller only)
`StartAddress`	Int32	r/w	Modelled attribute	Set/Get new StartAddress value.

Restriction

- Attribute `StartAddress`
  - Setting `StartAddress` may implicit change the `StartAddress` of the opposite I0 Type at the name module. Changing of input address changes the output address.
  - Writing access is not supported for all devices.
  - Packed addresses are not supported in TIA Portal Openness
  - Changing an address via TIA Portal Openness will not rewire the assigned tags.
- Attribute `InterruptObNumber`
  - Only accessible in settings with S7-300 or S7-400 controllers. Writing access is supported for S7-400 controllers.

Program code: Get or set attributes of an address object

Modify the following program code to access isochronous mode of an address object:

```
Address address= ...;
  // read attribute
bool attributeValue =
(bool)address.GetAttribute("IsochronousMode");
 // write attribute
address.SetAttribute("IsochronousMode", true);
```

Modify the following program code to access the `ProcessImage` attribute of an address object:

```
Address address= ...;
  // read attribute
int attributeValue =
(int)address.GetAttribute("ProcessImage");
 // write attribute
address.SetAttribute("ProcessImage", 7);
```

Modify the following program code to access the `InterruptObNumber` attribute of an address object:

```
Address address= ...;
  // read attribute
long attributeValue =
(long)address.GetAttribute("InterrruptObNumber");
 // write attribute
address.SetAttribute("InterrruptObNumber", 42L);
//default value = 40
```

Modify the following program code to access the `StartAddress` attribute of an address object:

```
Address address= ...;
  // read attribute
int attributeValue =
(int)address.GetAttribute("StartAddress");
 // write attribute
address.StartAddress = IntValueStartAddress;
```

Program code: Assign the current process image to an OB

Modify the following program code to assign the current process image to an OB:

```
OB obX =…
Address address= ...;
// assign PIP 5 to obX
address.SetAttribute("ProcessImage", 5);
try
{
        address.AssignProcessImageToOrganizationBlock(obX);
} catch(RecoverableException e) {
       Console.WriteLine(e.Message);
}
// remove this PIP-OB assignment
try
```

```
{
        address.AssignProcessImageToOrganizationBlock(null);
} catch(RecoverableException e) {
       Console.WriteLine(e.Message);
}
```

1.4 Accessing the channels of a module

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

Signal Modules like analog input modules usually have multiple channels within a single module. Usually, channels provide similar functionality multiple times, e.g. an analog input module with four channels can measure four voltage values at the same time.

To access all channels of a module, the Channels attribute of an device item is used.

Program code: Attributes of channels

Modify the following program code to access attributes of a channel:

```
Copies the following program code to clipboard.
DeviceItem aiModule = ...
ChannelComposition channels = aiModule.Channels;
foreach (Channel channel in channels)
{
    ... // Work with the channel
}
```

Program code: Identifying attributes

Modify the following program code to get the identifying attribute for each channel:

```
Copies the following program code to clipboard.
```

```
Channel channel = ...
int channelNumber = channel.Number;
ChannelType type = channel.Type;
ChannelIoType ioType = channel.IoType;
```

Program code: Accessing a single channel

Modify the following program code to use the identifying attributes to access a channel directly:

```
Copies the following program code to clipboard.
DeviceItem aiModule = ...
Channel channel = aiModule.Channels.Find(ChannelType.Analog,
ChannelIoType.Input, 0);
... // Work with the channel
```

Channel types

Value	Description
ChannelType.None	The channel type invalid.
ChannelType.Analog	The channel type is analog.
ChannelType.Digital	The channel type is digital.
ChannelType.Technology	The channel type is technology.

Channel IO types

Value	Description
ChannelIOType.None	The channel IO type invalid.
ChannelIOType.Input	An input channel.
ChannelIOType.Output	An output channel.
ChannelIOType.Complex	Complex IO types, e.g. for technological channels.

Functions on networks

2

2.1 Creating a subnet

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

Subnets can be created in two different ways:

- Create a subnet that is connected to an interface: The type of the interface, where the subnet is created, determines the type of the subnet
- Create a subnet not connected to an interface.

Program code: Create a subnet connected to a node

Modify the following program code to create a subnet:

```
Copies the following program code to clipboard.

Node node = ...;
Subnet subnet =
node.CreateAndConnectToSubnet("NameOfSubnet");
```

The following type identifiers are used:

- System:Subnet.Ethernet
- System:Subnet.Profibus
- System:Subnet.Mpi
- System:Subnet.Asi

Program code: Create a subnet not connected to an interface

Modify the following program code to create a subnet:

```
Copies the following program code to clipboard.
```

```
Project project = ...;
SubnetComposition subnets = project.Subnets;
Subnet newSubnet = subnets.Create("System:Subnet.Ethernet",
"NewSubnet");
```

The following type identifiers can be used:

- System:Subnet.Ethernet
- System:Subnet.Profibus
- System:Subnet.Mpi
- System:Subnet.Asi

2.2 Accessing subnets

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

For several network related features, e.g. assigning interfaces to a subnet, you have to access subnets in the project. Typically, subnets are aggregated directly on project level.

Program code: Access all subnets of a project

Modify the following program code to access all subnets excluding internal subnets of a project:




```
Project project = ...
foreach (Subnet net in project.Subnets)
{
    ... // Work with the subnet
}
```

Program code: Access a specific subnet

Modify the following program code to access a specific subnet by it's name:

```
Copies the following program code to clipboard.
Project project = ...
Subnet net = project.Subnets.Find("PROFIBUS_1");
{
    ... // Work with the subnet
}
```

Attributes of a subnet

A subnet has the following attributes:

```
Copies the following program code to clipboard.
Subnet net = ...;
string name = net.Name;
NetType type = net.NetType;
```

Network types

Value	Description
NetType.Unknown	The type of the network is unknown.
NetType.Ethernet	The type of the network is Ethernet.
NetType.Profibus	The type of the network is Profibus.
NetType.Mpi	The type of the network is MPI.
NetType.ProfibusIntegrated	The type of the network is integrated Profibus.
NetType.Asi	The type of the network is ASi.
NetType.PcInternal	The type of the network is PC internal.
NetType.Ptp	The type of the network is PtP.
NetType.Link	The type of the network is Link.
NetType.Wan	The type of the network is Wide Area Network

2.3 Accessing internal subnets

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

If a device item is able to compose a subnet, it provides the additional functionality "subnet owner". To access this additional functionality, a specific service of the device item must be used.

Program code: Get the subnet owner role

Modify the following program code to get the subnet owner role:

```
Copies the following program code to clipboard.

SubnetOwner subnetOwner =
((IEngineeringServiceProvider)deviceItem).GetService<SubnetOwner>
();
if (subnetOwner != null)
{
    // work with the role
}
```

Program code: Attributes of a subnet owner

Modify the following program code to access the subnets of a subnet owner:

```
Copies the following program code to clipboard.

foreach(Subnet subnet in subnetOwner.Subnets)
{
    Subnet interalSubnet = subnet;
}
```

2.4 Get type identifier of subnets

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

The attribute `TypeIdentifier` is used to identify a subnet. The `TypeIdentifier` is a string consisting of several parts: `<TypeIdentifierType>:<SystemIdentifier>`

Possible values for `TypeIdentifierType` are:

- `System`

SystemIdentifier

Subnet type	SystemIdentifier
PROFIBUS	Subnet.Profibus
MPI	Subnet.Mpi
Industrial Ethernet	Subnet.Ethernet
ASI	Subnet.Asi
Ptp	Subnet.Ptp
ProfibusIntegrated	Subnet.ProfibusIntegrated
PcInternal	Subnet.PcInternal
ProfidriveIntegrated	Subnet.ProfidriveIntegrated

Program code

Modify the following program code to get the type identifier for user manageable and separately creatable objects for GSD:



```
Subnet subnet = ...;
string typeIdentifier = subnet.TypeIdentifier;
```

2.5 Accessing attributes of a subnet

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Introduction

A subnet provides certain mandatory attributes that can be read and/or written. The attributes are only available, if they are available at the UI. Writing is generally only allowed if an attribute can also be changed by the user at the UI. This might vary depending on the type of the subnet. For example, the user can only set the DpCycleTime, if the IsochronousMode is true and the DpCycleMinTimeAutoCalculation is false

Attributes of subnets of type ASI

Attribute	Data type	Writable	Access	Description
Name	string	r/w	modeled	Name of the subnet.
NetType	NetType	r	modeled	Type of the subnet
SubnetId	string	r	dynamic	Unique identification of the subnet. The S7 subnet ID is made up of two numbers separated by a hyphen. One number for the project and one for the subnet. e.g. 4493-1.

Attributes of subnets of type Ethernet

Attribute	Data type	Writable	Access	Description
Name	string	r/w	modeled	Name of the subnet.

NetType	NetType	r	modeled	Type of the subnet
SubnetId	string	r/w	dynamic	Unique identification of the subnet. The S7 subnet ID is made up of two numbers separated by a hyphen. One number for the project and one for the subnet. e.g. 4493-1.
DefaultSubnet	bool	r/w	dynamic	true if the subnet is a default subnet. There is at most one default subnet in a project.

Attributes of subnets of type MPI

Attribute	Data type	Writable	Access	Description
Name	string	r/w	modeled	Name of the subnet.
NetType	NetType	r	modeled	Type of the subnet
SubnetId	string	r/w	dynamic	Unique identification of the subnet. The S7 subnet ID is made up of two numbers separated by a hyphen. One number for the project and one for the subnet. e.g. 4493-1.
HighestAddress	int	r/w	dynamic	Highest MPI address at subnet.
TransmissionSpeed	BaudRate	r/w	dynamic	True if the subnet is a default subnet. There is at most one default subnet in a project.

Attributes of subnets of type PC internal

Attribute	Data type	Writable	Access	Description
Name	string	r	modeled	Name of the subnet.
NetType	NetType	r	modeled	Type of the subnet

SubnetId	string	r	dynamic	Unique identification of the subnet. The S7 subnet ID is made up of two numbers separated by a hyphen. One number for the project and one for the subnet. e.g. 4493-1.

Attributes of subnets of type PROFIBUS

Attribute	Data type	Writable	Access	Description
Name	string	r/w	modeled	Name of the subnet.
NetType	NetType	r	modelled	Type of the subnet
SubnetId	string	r/w	dynamic	Unique identification of the subnet. The S7 subnet ID is made up of two numbers separated by a hyphen. One number for the project and one for the subnet. e.g. 4493-1.
HighestAddress	int	r/w	dynamic	Highest PROFIBUS address at subnet.
TransmissionSpeed	BaudRate	r/w	dynamic	True if the subnet is a default subnet. There is at most one default subnet in a project.
BusProfile	BusProfile	r/w	dynamic	The PROFIBUS profile.
PbCableConfiguration	bool	r/w	dynamic	True to enable additional PROFIBUS network settings
PbRepeaterCount	int	r/w	dynamic	Number of repeaters for copper cable

PbCopperCableLength	double	r/w	dynamic	The length of the copper cable
PbOpticalComponentCount	int	r/w	dynamic	Number of OLMs and OBTs of fiber-optical cable.
PbOpticalCableLength	double	r/w	dynamic	The length of the fiber-optical cable for the PROFIBUS network in km.
PbOpticalRing	bool	r/w	dynamic	True if bus parameter are adapted for an optical ring
PbOlmP12	bool	r/w	dynamic	True if OLM/P12 is enabled for bus parameter calculation
PbOlmG12	bool	r/w	dynamic	True if OLM/G12 is enabled for bus parameter calculation
PbOlmG12Eec	bool	r/w	dynamic	True if OLM/G12-EEC is enabled for bus parameter calculation
PbOlmG121300	bool	r/w	dynamic	True if OLM/G12-1300 is enabled for bus parameter calculation
PbAdditionalNetworkDevices	bool	r/w	dynamic	True if additional bus devices that don't exist in project will be taken in to account when calculating bus times.
PbAdditionalDpMaster	int	r/w	dynamic	Number of unconfigured DP masters.
PbTotalDpMaster	int	r	dynamic	Number of total DP masters
PbAdditionalPassiveDevice	int	r/w	dynamic	Number of unconfigured DP slaves or passive devices.
PbTotalPassiveDevice	int	r	dynamic	Number of total DP slaves or passive devices.

PbAdditionalActiveDevice	int	r/w	dynamic	Number of unconfigured active devices with FDL/FMS/S/ communication load.
PbTotalActiveDevice	int	r	dynamic	Number of total active devices with FDL/FMS/S/ communication load.
PbAdditionalCommunicationLoad	CommunicationLoad	r/w	dynamic	Rough quantification of the communication load
PbDirectDateExchange	bool	r/w	dynamic	Optimization for direct data exchange.
PbMinimizeTslotForSlaveFailure	bool	r/w	dynamic	Minimization for time allocation for slave failure.
PbOptimizeCableConfiguration	bool	r/w	dynamic	Optimiazation of the cable configuration.
PbCyclicDistribution	bool	r/w	dynamic	True if enable cyclic distribution of bus parameter.
PbTslotInit	int	r/w	dynamic	Default value of Tslot.
PbTslot	int	r	dynamic	Waiting to receive time (slot time)

PbMinTsdr	int	r/w	dynamic	Minimum protocol processing time
PbMaxTsdr	int	r/w	dynamic	Maximum protocol processing time
PbTid1	int	r	dynamic	Idle time 1
PbTid2	int	r	dynamic	Idle time 2
PbTrdy	int	r	dynamic	Ready time
PbTset	int	r/w	dynamic	Setup time
PbTqui	int	r/w	dynamic	Quiet time for modulator
PbTtr	int64	r/w	dynamic	The Ttr value in t_Bit
PbTtrTypical	int64	r	dynamic	Average response time on bus
PbWatchdog	int64	r/w	dynamic	Watchdog
PbGapFactor	int	r/w	dynamic	Gab update factor
PbRetryLimit	int	r/w	dynamic	Maximum number of retries
IsochronousMode	bool	r/w	dynamic	True if constant bus cycle time is enabled.

PbAdditionalPassivDeviceForIsochronousMode	int	r/w	dynamic	Number of additional OPs/PGs/TDs etc. that are not configured in this network view.
PbTotalPassivDeviceForIsochronousMode	int	r	dynamic	Sum of configured and unconfigured devices, such as OPs/PGs/TDs etc.
DpCycleMinTimeAutoCalculation	bool	r/w	dynamic	True if automatic calculation and setting of shortest DP cycle time is enabled.
DpCycleTime	double	r/w	dynamic	The DP cycle time.
IsochronousTiToAutoCalculation	bool	r/w	dynamic	True if automatic calculation and setting of values of IsochronousTi and IsochronousTo.
IsochronousTi	double	r/w	dynamic	Time Ti (read in process values)
IsochronousTo	double	r/w	dynamic	Time To (output process values)

Attributes of subnets of type PROFIBUS Integrated

Attribute	Data type	Writable	Access	Description

Name	string	r/w	modeled	Name of the subnet.
NetType	NetType	r	modeled	Type of the subnet
SubnetId	string	r/w	dynamic	Unique identification of the subnet. The S7 subnet ID is made up of two numbers separated by a hyphen. One number for the project and one for the subnet. e.g. 4493-1.
IsochronousMode	bool	r	dynamic	Enabled constant bus cycle time.
DpCycleMinTimeAutoCalculation	bool	r/w	dynamic	True if automatic calculation and setting of shortest DP cycle time is enabled.
DpCycleTime	double	r/w	dynamic	The DP cycle time.
IsochronousTiToAutoCalculation	bool	r/w	dynamic	True if automatic calculation and setting of values of IsochronousTi and IsochronousTo.
IsochronousTi	double	r/w	dynamic	Time Ti (read in process values)
IsochronousTo	double	r/w	dynamic	Time To (output process values)

Attributes of subnets of type PROFIdrive Integrated

Attribute	Data type	Writable	Access	Description
Name	string	r/w	dynamic	Name of the subnet
NetType	NetType	r	modelled	Type of the subnet

IsochronousMode	bool	r	dynamic	Enabled constant bus cycle time
SourceCycleTime*	Int32	r/w	dynamic	The bus source cycle time
DpCycleTime	double	r/w	dynamic	The DP cycle time
IsochronousTiToAutoCalculation	bool	r/w	dynamic	True if automatic calculation and setting of values of Ti (read in process values) and To (output process values)
IsochronousTi	double	r/w	dynamic	Time Ti (read in process values)
IsochronousTo	double	r/w	dynamic	Time To (output process values)

The *SourceCycleTime supports the following Enums value:

Enum Name	Value
Manual	0
AutomaticMinimum	1
LocalSendClock	2
ProfinetSendClock	3

Program code

Modify the following program code to get or set the attributes of a subnet:

```
 Copies the following program code to clipboard.

SubnetOwner subnetOwner =
((IEngineeringServiceProvider)deviceItem).GetService<SubnetOwner>
();

Subnet subnet = subnetOwner.Subnets[0];

string nameValue = subnet.Name;
```

```
NetType nodeType = (NetType)subnet.NetType;
string subnetId =
((IEngineeringObject)subnet).GetAttribute("SubnetId");
subnet.Name = "NewName";
subnet.SetAttribute("Name", "NewName");
bool isDefaultSubnet =
((IEngineeringObject)subnet).GetAttribute("DefaultSubnet");
```

Baud rates

Value	Description
BaudRate.None	The baud rate is unknown.
BaudRate.Baud9600	9.6 kBaud
BaudRate.Baud19200	19.2 kBaud
BaudRate.Baud45450	45.45 kBaud
BaudRate.Baud93700	93.75 kBaud
BaudRate.Baud187500	187.5 kBaud
BaudRate.Baud500000	500 kBaud
BaudRate.Baud1500000	1.5 MBaud
BaudRate.Baud3000000	3 MBaud
BaudRate.Baud6000000	6 MBaud
BaudRate.Baud12000000	12 MBaud

Bus profiles

Value	Description
BusProfile.None	The bus profile is unknown.

BusProfile.DP	The type of the network is DP.
BusProfile.Standard	The type of the network is Standard.
BusProfile.Universal	The type of the network is Universal.
BusProfile.UserDefined	The type of the network is user defined.

Communication load

Value	Description
CommunicationLoad.None	No valid communication load.
CommunicationLoad.Low	Typically used for DP, no great data communication apart from DP.
CommunicationLoad.Medium	Typically used for mixed operations featuring DP and other communication services, such as for S7 communication, when DP has strict time requirements and for average acyclic volumes of communication.
CommunicationLoad.High	For mixed operations featuring DP and other communication services, such as for S7 communication, when DP has loose time requirements and for high acyclic volumes of communication.

2.6 Deleting a global subnet

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Program code

Modify the following program code to delete a a global subnet within a project.:



```
Project project = ...;
SubnetComposition subnets = project.Subnets;
subnets.First();
// delete subnet
Subnet subnetToDelete = ...;
subnetToDelete.Delete();
```

2.7 Enumerate all participants of a subnet

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

Enumeration of all participants on a subnet.

Program code

Modify the following program code to enumerate dp master systems from subnet:

```
Copies the following program code to clipboard.
Subnet subnet = ...;
foreach (Node node in subnet.Nodes)
    {
        // work with the node
    }
```

2.8 Enumerate IO systems of a subnet

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

Enumeration of IoSystem provides all io systems that are present on a subnet. The class IoSystem represents the master systems and the io systems.

Program code

Modify the following program code to enumerate dp master systems from subnet:

```
Subnet subnet = ...;
foreach (IoSystem ioSystem in subnet.IoSystems)
    {
        // work with the io system
    }
```

2.9 Accessing nodes

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

The role interface aggregates nodes to access attributes that are related to the address and subnet assignment of an interface.

The name of a node can be seen in the attributes of an interface in TIA Portal . The NodeId is a unique identifier for every node aggregated at an interface, its value can only be seen via TIA Portal Openness.

Program code

Modify the following program code to access all nodes of an interface:

```
NetworkInterface itf = ...
foreach (Node node in itf.Nodes)
```

```
{
    ... // Work with the node
}
```

Most interfaces provide only a single node, therefore, usually the first node is used:

```
NetworkInterface itf = ...
Node node = itf.Nodes.First();
{
    ... // Work with the node
}
```

Nodes provide their names and types and NodeIds as attributes:

```
Node node = ...
string name = node.Name;
NetType type = node.NodeType;
string id = node.NodeId;
```

2.10 Accessing attributes of a node

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

A device item provides certain mandatory attributes that can be read and/or written. The attributes are only available if they are available at the UI. Writing is generally only allowed if an attribute can also be changed by the user at the UI. This might vary depending on the type of the device item. For example, the user can only set the RouterAddress if the RouterUsed is true. If the user changes the SubnetMask at IO controller, Subentmask on all IO devices will be also changed to the same value.

Attributes of a node of type ASI

Attributes	Data type	Writable	Access	Description
Name	string	r	modeled	Name of the node.
NodeId	string	r	modeled	ID of the node.
NodeType	NetType	r	modeled	A node gets his type from the subnet.
Address	string	r/w	dynamic	Additional attribute for AS-i slaves.

Attributes of a node of type Ethernet

Attributes	Data type	Writable	Access	Description
Name	string	r	modeled	Name of the node.
NodeId	string	r	modeled	ID of the node.
NodeType	NetType	r/w sometimes r/o	modeled	A node gets his type from the subnet.
UseIsoProtocol	bool	r/w	dynamic	True, if ISO protocol should be used
MacAddress	string	r/w	dynamic	e.g. 01-80-C2-00-00-00
UseIpProtocol	bool	r/w	dynamic	This value can be read even if it noct visible at the corresponding TIA UI control.
IpProtocolSelection	enum	r/w	dynamic	-
Address	string	r/w	dynamic	only IPv4 and no IPv6 is supported
SubnetMask	string	r/w	dynamic	-
UseRouter	bool	r/w	dynamic	-

RouterAddress	string	r/w	dynamic	-
DhcpClientId	string	r/w	dynamic	-
PnDeviceNameSetDirectly	bool	r/w	dynamic	PROFINET device name is set directly at the device. Not available for every device.
PnDeviceNameAutoGeneration	bool	r/w	dynamic	PROFINET device name is created automatically.
PnDeviceName	string	r/w	dynamic	Unique device name in subnet.
PnDeviceNameConverted	string	r	dynamic	Device name converted for system internal using.

Attributs of a node of type MPI

Attribut	Data type	Writable	Access	Description
Name	string	r	modeled	Name of the node.
NodeId	string	r	modeled	ID of the node.
NodeType	NetType	r	modeled	A node gets his type from the subnet.
Address	string	r/w	dynamic	-

Attributs of a node of type PC internal

Attribut	Data type	Writable	Access	Description
Name	string	r	modeled	Name of the node.
NodeId	string	r	modeled	ID of the node.
NodeType	NetType	r	modeled	A node gets his type from the subnet.

Attributs of a node of type PROFIBUS

Attribut	Data type	Writable	Access	Description
Name	string	r	modeled	Name of the node.
NodeId	string	r	modeled	ID of the node.
NodeType	NetType	r	modeled	A node gets his type from the subnet.
Address	string	r/w	dynamic	The network address of the node. The type of address depends on the node type (e.g. IP address for PROFINET nodes, PROFIBUS address for PROFIBUS nodes)

Attributs of a node of type PROFIBUS Integrated

Attribute	Data type	Writable	Access	Description
Name	string	r	modeled	Name of the node.
NodeId	string	r	modeled	ID of the node.
NodeType	NetType	r	modeled	A node gets his type from the subnet.
Address	string	r	dynamic	-

Attributs of a node of type PROFIdrive Integrated

Attribute	Data type	Writable	Access	Description
Address	string	r	dynamic	The network address of the node.
ConnectedSubnet	Subnet	r	modelled	The connected subnet.

Name	string	r	modelled	Name of the node.
NodeId	string	r	modelled	ID of the node.
NodeType	NetType	r	modelled	A node gets its type from the subnet.

Program code: Attributes of a node

Modify the following program code to get or set the attributes of a node:



```
Copies the following program code to clipboard.
Node node = ...;
string nameValue = node.Name;
NetType nodeType = (NetType)node.NodeType;
node.NodeType = NetType.Mpi;
```

Program code: Dynamic attributes

Modify the following program code to get or set dynamic node attributes:



```
Copies the following program code to clipboard.
Node node = ...;
var attributeNames = new[]
{
    "Address", "SubnetMask", "RouterAddress", "UseRouter",
"DhcpClientId", "IpProtocolSelection"
};
foreach (var attributeName in attributeNames)
{
    object attributeValue =
((IEngineeringObject)node).GetAttribute(attributeName);
}
```

Protocol selection

Value	Description

IpProtocolSelection.None	Error value
IpProtocolSelection.Project	IP suite configured within project.
IpProtocolSelection.Dhcp	IP suite managed via DHCP protocol. DHCP Client ID necessary.
IpProtocolSelection.UserProgram	IP suite set via FB (function block).
IpProtocolSelection.OtherPath	IP suite set via other methods, for example PST tool.
IpProtocolSelection.ViaIoController	IP suite set via IO Controller in runtime.

Net type

Value	Description
NetType.Asi	Net type is ASI.
NetType.Ethernet	Net type is Ethernet.
NetType.Link	Net type is Link.
NetType.Mpi	Net type is MPI.
NetType.PcInternal	Net type is PC internal.
NetType.Profibus	Net type is PROFIBUS.
NetType.ProfibusIntegrated	Net type is PROFIBUS Integrated.
NetType.ProfidriveIntegrated	Net type is PROFIdrive Integrated.
NetType.Ptp	Net type is PTP.
NetType.Wan	Net type is Wide Area Network (WAN).
NetType.Unknown	Net type is Unknow.

2.11 Connecting a node to a subnet

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Program code

Modify the following program code to assign a node (device, interface) to a network:

```
Node node = ...;
Subnet subnet = ...;
node.ConnectToSubnet(subnet);
```

2.12 Disconnect a node from a subnet

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Program code

Modify the following program code to disconnect a node (device, interface) from a network:

```
Node node = ...;
node.DisconnectFromSubnet();
```

2.13 Creating an IO system

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Introduction

An io system is created by calling the action IoController.CreateIoSystem("name") on an object of the type IoController. In case name is null or String.Empty the default name will be used. The io controller is aquired by accessing the attribute IoControllers object on the NetworkInterface. The IoControllers navigator returns one IoController object.

Prerequisites for creating an io system:

- The interface of the io controller is connected to a subnet.
- The io controller has no io system.

Program code

Modify the following program code to create an io system:

```
Copies the following program code to clipboard.
using System.Linq;
//...
NetworkInterface interface12 = ...;
IoSystem ioSystem = ...;
// Interface is configured as io controller
if((interface12.InterfaceOperatingMode &
InterfaceOperatingModes.IoController) != 0)
{
IoControllerComposition ioControllers =
interface12.IoControllers;
IoController ioController = ioControllers.First();
if(ioController != null)
{
ioSystem = ioController.CreateIoSystem("io system");
}
}
```

2.14 Accessing the attributes of an IO system

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

See Connecting to the TIA Portal

- A project is open.

  See Opening a project

Application

The master system and the io system will both be represented by the class IoSystem.

Program code: Attributes of an io system

Modify the following program code to get the attributes of the IoSystem:

```
Copies the following program code to clipboard.
NetworkInterface itf = ...
foreach (IIoController ioController in itf.IoControllers)
{
    IoSystem ioSystem = ioController.IoSystem;
    int ioSystemNumber = ioSystem.Number;
    string ioSystemName = ioSystem.Name;
}
```

IntroductionProgram code: Subnet of an io system

Modify the following program code to navigate to the subnet the io system is assigned to:

```
Copies the following program code to clipboard.
Subnet subnet = ioSystem.Subnet;
```

2.15 Connecting an IO connector to an IO system

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal

- A project is open.

  See Opening a project

Application

Use the action ConnectToIoSystem(IoSystem ioSystem) of IoConnector to connect a profinet or a DP IoConnector to an existing io system.

Use the action GetIoController to navigate to the remote IoController. For further information how to navigate to the local IoConnector and the io system see Get master system or IO system of an interface.

Prerequisites:

- The IoConnector is not yet connected to an io system.
- The IoConnector interface is connected to the same subnet as the interface of the desired IoController.

Program code

Modify the following program code:

```
Copies the following program code to clipboard.
NetworkInterface itf = ...;
IoSystem ioSystem = null;
foreach (IoController ioController in itf.IoControllers)
{
ioSystem = ioController.IoSystem;
IoConnector ioConnector = itf.IoConnectors[0];
ioConnector.ConnectToIoSystem();
IoController ioController2 = ioConnector.GetIoController();
}
```

2.16 Get master system or IO system of an interface

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

The service NetworkInterface provides the navigator IoControllers, each IoController in turn provides the navigator IoSystem. The class IoSystem represents the master systems and the io systems. The io device and the slave are both named io device.

- The IoControllers navigator returns IoController objects, if the network interface can have an io system. At the moment only one io controller will be returned.
- The IoConnectors navigator returns IoConnector objects, if the network interface can be connected to an io system as an io device. At the moment only one io connector will be returned.

Program code: Get the io system of the IoController

Modify the following program code to get the io system of the IoController:

```
Copies the following program code to clipboard.
NetworkInterface itf = ...
foreach (IoController ioController in itf.IoControllers)
{
    IoSystem ioSystem = ioController.IoSystem;
    // work with the io system
}
```

Program code: Get the io system of the IoConnector

Modify the following program code to get the io system of the IoConnector:

```
Copies the following program code to clipboard.
NetInterface itf = ...
foreach (IoConnector ioConnector in itf.IoConnectors)
{
    IoSystem ioSystem = ioConnector.ConnectedIoSystem;
    // work with the io system
}
```

2.17 Get an IO Controller

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Program code

Currently only configurations with one IoController are possible. An IoController does not provide any modelled attributes or actions.

Program code

Modify the following program code to get the io controller:

```
Copies the following program code to clipboard.
NetworkInterface itf = ...
foreach (IoController ioController in itf.IoControllers)
{
    // work with the io controller
}
```

2.18 Get an IO Connector

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

An IoConnector does provide the modelled attributes or actions.

The following attribute and actions are available at the IoConnector:

Actions

Action	Signature	Description
ConnectToIoSystem	void ConnectToIoSystem(Siemens.Engineering.HW.IoSystem ioSystem	To connect an IoConnector to an existing DP master system

DisconnectFromIoSystem	void DisconnectFromIoSystem()	To disconnect an IoConnector from an existing io system
GetIoController	Siemens.Engineering.HW.IoController GetIoController()	Returns the IO controller for this connector

Links

Link	Type	Access
ConnectedToIoSystem	Siemens.Engineering.HW.IoSystem	read-only

Program code

Modify the following program code to get the io connector:

```
Copies the following program code to clipboard.
NetworkInterface itf = ...
foreach (IoConnector ioConnector in itf.IoConnectors)
{
    // work with the IoConnector
}
```

2.19 Disconnecting an IO connector from an IO system or a DP mastersystem

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

Use the action DisconnectFromIoSystem() of IoConnector to disconnect an IoConnector from an existing io system or an existing DP mastersystem.

For further information how to navigate to the local IoConnector and the io system see Get master system or IO system of an interface.

Program code

Modify the following program code:

```
Copies the following program code to clipboard.
IoSystem ioSystem = ...;
IoConnector ioConnector = ...;
ioConnector.DisconnectFromIoSystem();
```

2.20 Accessing attributes of a DP mastersystem

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Introduction

A DP Mastersystem provides certain attributes that can be read and/or written. The attributes are only available if they are available at the TIA Portal. Writing is generally only allowed if an attribute can also be changed by the user at the TIA Portal. This might vary depending on the DP Master and the DP Slaves which are assigned to this DP Mastersystem.

Attributes of a dp mastersystem

Attribute	Data type	Writable	Access	Description
Name	string	r/w	property	-
Number	int	r/w	property	The property Number accepts values that cannot be set via UI. In this case the compile will fail.

Program code: Get attributes

Modify the following program code to get attributes:

```
Copies the following program code to clipboard.
IoSystem dpMastersystem = ...;
string name = dpMastersystem.Name;
int number = dpMastersystem.Number;
```

Program code: Set attributes

Modify the following program code to set attributes:

```
Copies the following program code to clipboard.
IoSystem dpMastersystem = ...;
dpMastersystem.Name ="myDpMastersystem"
dpMastersystem.Number=42;
```

2.21 Accessing attributes of PN Driver

Application

You can use the TIA Portal Openness to access the attributes in PROFINET Interface of PN Driver V2.2.

Access to attribute of PN Driver V2.2 submodule object

The following attributes can be accessed at the Interface options using TIA Portal Openness:

Openness Attribute	Type	ReadOnly
DeviceReplacementWithoutExhangeableMedium	Boolean	False
OverwriteDeviceNames	Boolean	False
IECV22LLDPMode	Boolean	False

The following attributes can be accessed at the Real time options using TIA Portal Openness:

Openness Attribute	Type	ReadOnly
CalculatedBandwidthForCyclicIoData	Int32	True
MaxBandwidthForCyclicIoData	Int32	True

The following attributes can be accessed at Isochronous mode options using TIA Portal Openness:

Openness Attribute	Type	ReadOnly
ApplicationCycle	Int64	False
AutomaticMinimum	Boolean	False
DelayTime	Double	False

Note

The value of DelayTime cannot be set, unless the value of the AutomaticMinimum is set to False beforehand.

2.22 Accessing attributes of a profinet io system

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

An IO System provides certain attributes that can be read and/or written. The attributes are only available if they are available at the UI. Writing is generally only allowed if an attribute can also be changed by the user at the UI. This might vary depending on the IO Controller and the IO Devices which are assigned to this IO System.

Attributes of a PROFINET io System

Attribute	Data type	Writable	Access	Description
MultipleUseIoSystem	bool	r/w	dynamic	-
Name	string	r/w	property	-
Number	int	r/w	property	The attribute Number accepts values that cannot be set via

				UI. In this case the compile will fai
UseIoSystemNameAsDeviceNameExtension	bool	r/w	dynamic	If MultipleUseIoSystem is set to TR UseIoSystemNameAsDeviceName will be set to FALSE and write acce possible.
MaxNumberIWlanLinksPerSegment	int	r/w	dynamic	-

Program code: Get attributes

Modify the following program code to get attributes

```
Copies the following program code to clipboard.

IoSystem ioSystem = ...;
string name = ioSystem.Name;
```

Program code: Set attributes

Modify the following program code to set attributes:

```
Copies the following program code to clipboard.

IoSystem ioSystem = ...;
ioSystem.Name = "IOSystem_1";
```

Program code: Get attributes with dynamic access

Modify the following program code to get the values of dynamic attributes:

```
Copies the following program code to clipboard.

IoSystem ioSystem = ...;
var attributeNames = new[]
{
    "MultipleUseIoSystem",
"UseIoSystemNameAsDeviceNameExtension",
"MaxNumberIWlanLinksPerSegment"
};
foreach (var attributeName in attributeNames)
{
```

```
    object attributeValue =
((IEngineeringObject)ioSystem).GetAttribute(attributeName);
}
```

Program code: Set attributes with dynamic access

Modify the following program code to set the values of dynamic attributes:

```
IoSystem ioSystem = ...;
((IEngineeringObject)ioSystem).SetAttribute("MultipleUseIoSystem",
true);
```

2.23 Deleting a DP master system

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Program code: Deleting a PROFINET io system

Modify the following program code to delete a PROFINET io system:

```
IoController ioController = ...;
IoSystem ioSystem = ioController.IoSystem;
ioSystem.Delete();
```

2.24 Deleting a profinet io system

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Program code

Modify the following program code to delete a profinet io system:

```
Copies the following program code to clipboard.
IoController ioController = ...;
IoSystem ioSystem = ioController.IoSystem;
ioSystem.Delete();
```

2.25 Creating a DP master system

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Introduction

A DP master system is created by calling the action CreateIoSystem(string nameOfIoSystem) on an object of the type IoController. The io controller is aquired by accessing the attribute IoControllers object on the NetworkInterface.

Prerequisites for creating a DP master system:

- The interface of the io controller is connected to a subnet.
- The io controller has no io system.

Program code

Modify the following program code to create a dp master system:

```
Copies the following program code to clipboard.
//...
NetworkInterface interface12 = ...;
IoSystem dpMasterSystem = ...;
// Interface is configured as master or as master and slave
if((interface12.InterfaceOperatingMode &
InterfaceOperatingModes.IoController) != 0)
{
```

```
IoControllerComposition ioControllers =
interface12.IoControllers;
IoController ioController = ioControllers.First();
if(ioController != null)
{
dpMasterSystem = ioController.CreateIoSystem("dp master
system");
}
}
```

2.26 Accessing port interconnection information of port device item

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

NetworkPort provides the link ConnectedPorts which is an enumeration of ports to access all interconnected partner ports of a port.

It is only possible to interconnect ports which can also be interconnected in the TIA UI, e.g. it is not possible to interconnect two ports of the same Ethernet interface. Recoverable exception are thrown

- if there is already an interconnection to the same partner port
- when trying to interconnect two ports which cannot be interconnected
- when trying to create a second interconnection to a port which does not support alternative partners

Program code: Get the port interconnection

Modify the following program code to get the port interconnection information of a port device item:

```
 Copies the following program code to clipboard.
NetworkPort port = ...;
foreach (NetworkPort partnerPort in port.ConnectedPorts)
```

```
{
    // work with the partner port
}
```

Program code: Create port interconnections

Modify the following program code:

```
NetworkPort port1 = ...;
NetworkPort port2 = ...;
port1.ConnectToPort(port2);
// port supports alternative partners
NetworkPort port1 = ...;
NetworkPort port2 = ...;
NetworkPort port3 = ...;
port1.ConnectToPort(port2);
port1.ConnectToPort(port3);
```

Program code: Delete port interconnection

Modify the following program code:

```
NetworkPort port1 = ...;
NetworkPort port2 = ...;
port1.DisconnectFromPort(port2);
```

2.27 Attributes of port inter-connection

Attributes for port inter-connections

TIA Portal Openness supports the following attributes for port inter-connections. If the attributes are available in UI, the corresponding attributes can also be accessed through TIA Portal Openness. If the user has access to modifiy the attributes in UI, then they can also be modified via TIA Portal Openness.

Attribute name	Data type	Writable	Access	Description

MediumAttachmentType	MediumAttachmentType	Read-only	Dynamic attribute	-
CableName	CabelName	Read-Write	Dynamic attribute	-
AlternativePartnerPorts	Boolean	Read-Write	Dynamic attribute	Only available if toolchanger functionality is supported.
SignalDelaySelection	SignalDelaySelection	Read-Write	Dynamic attribute	-
CableLength	CableLength	Read-Write	Dynamic attribute	-
SignalDelayTime	Double	Read-Write	Dynamic attribute	-

Enum values of port inter-connection attributes

The Enum MediumAttachmentType has following values.

Value	Description
MediumAttachmentType.None	Attachment type cannot be determined.
MediumAttachmentType.Copper	Attachment type is copper.
MediumAttachmentType.FiberOptic	Attachment type is fiber optic.

The Enum CableName has following value

Value	Description
CableName.None	No cable name is specified
CableName.FO_Standard_Cable_9	FO standard cable GP (9 µm)
CableName.Flexible_FO_Cable_9	Flexible FO cable (9 µm)

CableName.FO_Standard_Cable_GP_50	FO standard cable GP (50 µm)
CableName.FO_Trailing_Cable_GP	FO trailing cable / GP
CableName.FO_Ground_Cable	FO ground cable
CableName.FO_Standard_Cable_62_5	FO standard cable (62.5 µm)
CableName.Flexible_FO_Cable_62_5	Flexible FO cable (62.5 µm)
CableName.POF_Standard_Cable_GP	POF standard cable GP
CableName.POF_Trailing_Cable	POF trailing cable
CableName.PCF_Standard_Cable_GP	PCF trailing cable / GP
CableName.GI_POF_Standard_Cable	GI-POF standard cable
CableName.GI_POF_Trailing_Cable	GI-POF trailing cable
CableName.GI_PCF_Standard_Cable	GI-PCF standard cable
CableName.GI_PCF_Trailing_Cable	GI-PCF trailing cable
CableName.GI_POF_Standard_Cable	GI-POF standard cable
CableName.GI_POF_Trailing_Cable	GI-POF trailing cable
CableName.GI_PCF_Standard_Cable	GI-PCF standard cable
CableName.GI_PCF_Trailing_Cable	GI-PCF trailing cable

The Enum SignalDelaySelection has following values.

Value	Description
SignalDelaySelection.None	-
SignalDelaySelection.CableLength	CableLength is used to define the signal delay.
SignalDelaySelection.SignalDelayTime	SignalDelayTime is used to define the signal delay

The Enum CableLength has following values

Value	Description
CableLength.None	CableLength is not specified.
CableLength.Length20m	Cable length is 20m.
CableLength.Length50m	Cable length is 50m.
CableLength.Length100m	Cable length is 100m.
CableLength.Length1000m	Cable length is 1000m.
CableLength.Length3000m	Cable length is 3000m.

Attributes of port options

The attributes of port options are given below.

Attribute name	Data type	Writable	Access
PortActivation	bool	Read-Write	Dynamic attribute
TransmissionRateAndDuplex	TransmissionRateAndDuplex	Read-Write	Dynamic attribute
PortMonitoring	bool	Read-Write	Dynamic attribute
TransmissionRateAutoNegotiation	bool	Read-Write	Dynamic attribute
EndOfDetectionOfAccessibleDevices	bool	Read-Write	Dynamic attribute
EndOfTopologyDiscovery	bool	Read-Write	Dynamic attribute
EndOfSyncDomain	bool	Read-Write	Dynamic attribute

The Enum TransmissionRateAndDuplex has following values.

Value	Description
TransmissionRateAndDuplex.None	-
TransmissionRateAndDuplex.Automatic	Automatic
TransmissionRateAndDuplex.AUI10Mbps	10 Mbps AUI
TransmissionRateAndDuplex.TP10MbpsHalfDuplex	TP 10 Mbps half duplex
TransmissionRateAndDuplex.TP10MbpsFullDuplex	TP 10 Mbps full duplex
TransmissionRateAndDuplex.AsyncFiber10MbpsHalfDuplex	async fiber 10Mbit/s half duplex mode
TransmissionRateAndDuplex.AsyncFiber10MbpsFullDuplex	async fiber 10Mbit/s full duplex mode
TransmissionRateAndDuplex.TP100MbpsHalfDuplex	TP 100 Mbps half duplex
TransmissionRateAndDuplex.TP100MbpsFullDuplex	TP 100 Mbps full duplex
TransmissionRateAndDuplex.FO100MbpsFullDuplex	FO 100 Mbps full duplex
TransmissionRateAndDuplex.X1000MbpsFullDuplex	X1000 Mbps full Duplex
TransmissionRateAndDuplex.FO1000MbpsFullDuplexLD	FO 1000 Mbps full duplex LD
TransmissionRateAndDuplex.FO1000MbpsFullDuplex	FO 1000 Mbps full Duplex
TransmissionRateAndDuplex.TP1000MbpsFullDuplex	TP 1000 Mbps full duplex
TransmissionRateAndDuplex.FO10000MbpsFullDuplex	FO 10000 Mbps full Duplex
TransmissionRateAndDuplex.FO100MbpsFullDuplexLD	FO 100 Mbps full duplex LD
TransmissionRateAndDuplex.POFPCF100MbpsFullDuplexLD	POF/PCF 100 Mbps full duplex

2.28 Accessing the attributes of a port

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  SeeConnecting to the TIA Portal
- A profect is open.

  See Opening a project

Application

If a device item is a port, it provides additional functionality over a simple device item.

- It is possible to access the linked partner ports of the port
- It is possible to access the interface of the port

To access this additional functionality, the NetworkPort feature, a specific service of the device item, must be used.

Program code: Accessing a port

Modify the following program code to access attributes of a channel:

```
Copies the following program code to clipboard.
NetworkPort port =
((IEngineeringServiceProvider)deviceItem).GetService<NetworkPort>
();
if (port != null)
{
    ... // Work with the port
}
```

Attributes of a port

A port has the following attributes:

```
Copies the following program code to clipboard.
NetworkPort port = ...;
var connectedPorts = port.ConnectedPorts;
var myInterface = port.Interface;
```

2.29 Enumerate DP master systems of a subnet

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

Enumeration of IoSystem provides all DP mastersystems that are present on a subnet. The class IoSystem represents the master systems and the io systems.

Program code

Modify the following program code to enumerate DP master systems from subnet:

```
Subnet subnet = ...;
foreach (IoSystem ioSystem in subnet.IoSystems)
    {
        // work with the io system
    }
```

2.30 Enumerate assigned IO connectors

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

The class IoSystem represents the master systems and the io systems.

It is used for:

- Enumeration of io connectors of a dp mastersystem
- Enumeration of io connectors of a profinet io system

Program code

Modify the following program code to enumerate assigned io connectors of the dp mastersystem:

```
Copies the following program code to clipboard.

IoSystem ioSystem = ...;
foreach (IoConnector ioConnector in
ioSystem.ConnectedIoDevices)
    {
        // work with the io connector
    }
```

2.31 Connecting a DP IO connector to a DP master system

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

Use the action ConnectToIoSystem(IoSystem ioSystem) of IoConnector to connect an IoConnector to an existing DP mastersystem.

Use the action GetIoController to navigate to the remote IoController. For further information how to navigate to the local IoConnector and the io system see Get master system or IO system of an interface.

Prerequisites:

- The IoConnector is not yet connected to an io system.
- The IoConnector interface is connected to the same subnet as the interface of the desired IoController.

Program code

Modify the following program code:

```
Copies the following program code to clipboard.

IoSystem ioSystem = ...;
IoConnector ioConnector = ...;
```

```
ioConnector.ConnectToIoSystem(ioSystem);
IoController ioController = ioConnector.GetIoController();
```

2.32 Accessing AS-i profile and parameter attributes for virtual slaves

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

The TIA Portal Openness supports the following additional AS-i profile parameters for the Virtual Slaves of the CTT5 AS-i Slave using the StructuredData names:

Name	Description
AsiProfileVirtualSlave1	This contains the AS-i profile parameters for the Virtual Slave 1
AsiProfileVirtualSlave2	This contains the AS-i profile parameters for the Virtual Slave2
AsiProfileVirtualSlave3	This contains the AS-i profile parameters for the Virtual Slave 3

The AS-i slave parameters for the Virtual slaves can be found below:

Name	Description
AsiParameterVirtualSlave1	This contains the AS-i parameter for the Virtual Slave 1
AsiParameterVirtualSlave2	This contains the AS-i parameter for the Virtual Slave 2
AsiParameterVirtualSlave3	This contains the AS-i parameter for the Virtual Slave 3

Program code

Modify the following program code to get and set the additional attributes of AS-i Slaves:



```
Copies the following program code to clipboard.

DeviceItem slaveModule = ...;

var structuredDataNamesProfile = new[]
{"AsiProfileVirtualSlave1", "AsiProfileVirtualSlave2",

"AsiProfileVirtualSlave3"};

var structuredDataNamesAsiParameter = new[]
{"AsiParameterVirtualSlave1", "AsiParameterVirtualSlave2",

"AsiParameterVirtualSlave3"};

var attributeNamesProfile = new[]{ "AsiProfileID",
"AsiProfileIO", "AsiProfileID2", "AsiProfileID1" };


string attributeNameAsiParameter = "AsiSlaveParameter";

foreach (var structuredDataName in
structuredDataNamesProfile)

{

foreach (var attributeName in attributeNamesProfile)

{

StructuredData structuredData =
(StructuredData)slaveModule.GetAttribute(structuredDataName);


//get

UInt32 attributeValue =
(UInt32)structuredData.GetAttribute(attributeName);

}

//set

slaveModule.SetAttribute(attributeName, (UInt32)5);

}

foreach (var structuredDataName in
structuredDataNamesAsiParameter)

{

StructuredData structuredData =
(StructuredData)slaveModule.GetAttribute(structuredDataName);

//get
```

```
UInt32 attributeValue =
(UInt32)structuredData.GetAttribute(attributeNameAsiParameter);
//set
slaveModule.SetAttribute(attributeName, (UInt32)5);
```

See also

Connecting to the TIA Portal
Opening a project

2.33 Accessing CM DP as DP slave and transfer area

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to configure an ET200SP PLC as DP slave if a CM DP has been plugged. It is also possible in TIA Portal Openness to create, configure and delete transfer areas at the DP interface. The handling will be similar to the handling of transfer areas at PN interface described in Openness transfer areas for PnPn coupler

The following dynamic attributes are supported at the device item DP interface.

Attribute name	Data type	Access	Writeable
DpUseForTestCommissioningRouting	Boolean	only available if configured as DP slave	r/w
DpWatchdog	Boolean	only available if configured as DP slave and assigned to DP Master	r/w

Program code: Creating transfer areas

Modify the following program code to create a transfer area at the DP interface:

```
Copies the following program code to clipboard.
NetworkInterface cpuItf =
CpuInterface.GetService<NetworkInterface>();
// Create TransferAreas
TransferAreaComposition transferAreas = cpuItf.TransferAreas;
// Simple TranferArea
TransferArea transferAreaExample =
transferAreas.Create("Example TA MS", TransferAreaType.MS);
```

Parameter of transfer areas

The following parameters are supported in transfer areas of DP slave configuration:

Parameter name	Data type	Access
Name	string	read/write
Direction	enum	read/write
Comment	string	read/write
LocalToPartnerLength	Int32	read/write
PartnerToLocalLength	Int32	read/write
LocalAddresses	AddressComposition object	read
PartnerAddresses	AddressComposition object	read
PositionNumber	Int32	read
Type	enum	read

Program code: Deleting transfer area

Modify the following program code to delete transfer area for DP slave configuration:

```
Copies the following program code to clipboard.
TransferArea transferAreaExample =
transferAreas.Create("Example TA MS", TransferAreaType.MS);
```

```
transferAreaExample.Delete();
```

See also

Opening a project

2.34 Coupling 1516 isochron central and decentral Profinet

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to perform the following operations:

- Given a 1516 PLC with one isochron capable module and it is possible to set the PLC into isochronous mode
- Given the previous configuration where the module is assigned to OB61 and it is possible to set the module into isochronous mode
- Given the previous config with an additional ET200SP assigned to the PLC and a module configured in isochronous mode assigned to the same OB61 where the source of sendclock can be set (PLC and decentral are coupled, Source of sendclock is set to PN X1).

Attributes

Attribute name	Data type	Writeable	Available range/values	Prerequisites	Description	Access
IsochronousMode	bool	r/w	True/False	Isochronous capable device	get/set Isochronus mode	Dynamic attribute
SourceCycleTime	int	r/w	Device specific	Isochronous mode is enabled	get/set Source of sendclock	Dynamic attribute
SendClock	float	r/w	Device specific	Isochronous mode is enabled	get/set Send clock	Dynamic attribute

IsochronousTiToCalculationMode	enum	r/w	FromOB, AutomaticMinimum, Manual	Isochronous mode is enabled	get/set Ti/To values	Dyn attri
IsochronousTi	double	r/w	Device specific	Isochronous mode is enabled	get/set Time Ti	Dyn attri
IsochronousTo	double	r/w	Device specific	Isochronous mode is enabled	get/set Time To	Dyn attri

Program code: Examples with S71500/ET200MP station

Modify the following program code for PLC represents an S71500/ET200MP station:

```
Copies the following program code to clipboard.
//plc represents an S71500/ET200MP station
//Getting headmodule of the station
DeviceItem plcHeadModule = plc.DeviceItems[1];
//Getting and I/O module of the station
DeviceItem module = plc.DeviceItems[2];
```

Modify the following program code to set isochoronous mode properties on PLC headmodule:

```
Copies the following program code to clipboard.
//Turn on isochronous mode
plcHeadModule.SetAttribute("IsochronousMode", true);
//Set source of sendclock to "Use send clock of PROFINET
interface [X1]"
plcHeadModule.SetAttribute("SourceCycleTime", 1);
//Set source of sendclock to "Local send clock"
plcHeadModule.SetAttribute("SourceCycleTime", 0);
//Set sendclock
plcHeadModule.SetAttribute("SendClock", (float)2.1);
//Set calculation mode to Automatic minimum
plcHeadModule.SetAttribute("IsochronousTiToCalculationMode",
IsochronousTiToCalculationMode.AutomaticMinimum);
```

```
//Set calculation mode to Manual
plcHeadModule.SetAttribute("IsochronousTiToCalculationMode",
IsochronousTiToCalculationMode.Manual);
//Set Time Ti
plcHeadModule.SetAttribute("IsochronousTi", (double)0.55);
//Set Time To
plcHeadModule.SetAttribute("IsochronousTo", (double)0.65);
```

Modify the following program code to set isochoronous mode properties on I/O modules:



```
//Isochronous relevant properties can be found on the Address
of the submodule
DeviceItem subModule = module.DeviceItems[0];
Address address = subModule.Addresses[0];
//Turn on isochronous mode
address.SetAttribute("IsochronousMode", true);
//Set Process image to 3
address.SetAttribute("ProcessImage", 3);
```

Program code: SIMATIC Drive Controller



For Setting isochoronous mode properties on PLC headmodule, It is same as in case of S71500/ET200MP station.

Modify the following program code to set isochoronous mode properties of central module (X142):



Modify the following program code to set isochoronous mode properties of an integrated telegram:



2.35 Openness for CP 1604/CP 1616/CP 1626

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- To compile the project, all devices must be "offline".

Introduction

You can use the TIA Portal Openness application to configure transfer areas and transfer area mapping rules for the communication processors CP 1604/CP 1616 as of V2.8 (also as of V2.7 depending on the article number) and CP 1626 as of V1.1.

Configuration of transfer areas

Creating transfer areas

To create a "CD" type transfer area for a CP 1604 modify the following program code:

```
Copies the following program code to clipboard.
NetworkInterface cpItf = CP
1604Interface.GetService<NetworkInterface>();
//Create Transfer Areas
TransferAreaComposition transferAreas = cpItf.TransferAreas;
//Simple TranferArea of type Input
TransferArea transferAreaInput =
transferAreas.Create("Input CD", TransferAreaType.CD);
```

The following attributes are used for creating transfer areas:

Attribute	Description
name	Specifies the name of the transfer area to be created.

type	Specifies the type of the transfer area to be created. The following types are possible:	
	TransferAreaType.CD	Data exchange controller device
	TransferAreaType.F_CD	Data exchange fail safe controller device
	TransferAreaType.F_PS	Data exchange PROFIsafe
	TransferAreaType.TM	Transfer module mapping
	Note: It is not possible to change the type at a later date.	

Setting attributes of the transfer areas

To set attributes of a transfer area modify the following program code:

```
 Copies the following program code to clipboard.
transferAreaTm.LocalToPartnerLength = 8;
transferAreaTm.Direction = TransferAreaDirection.LocalToPartner;
string name = transferAreaTm.Name
```

Some attributes must be set or queried, but all of them can be set or queried using the general calls "GetAttribute()" or "SetAttribute()" modify the following program code:

```
 Copies the following program code to clipboard.
const string myIndividualComment = "MyIndividualComment";
transferAreaTm.SetAttribute("Comment", myIndividualComment);
Int32 updateTime =
transferAreaTm.GetAttribute("TransferUpdateTime");
```

The following attributes can be set for transfer areas:

Attribute	Description
Name (string)	Specifies the name of the transfer area.

Direction	Specifies the direction in which the data of the transfer area is transferred. The following directions are possible:	
	TransferAreaDirection.LocalToPartner	Data of the transfer area is transferred from the IO device to the higher-level IO controller.
	TransferAreaDirection.partnerToLocal	Data of the transfer area is transferred from the higher-level IO controller to the IO device.
	TransferAreaDirection.bidirectional	Data of the transfer area can be transferred in both directions between the higher-level IO controller and the IO device. The "LocalToPartnerLength" attribute determines the length of the transferred data from IO device to the higher-level IO controller. The "PartnerToLocalLength" attribute determines the amount of data from the higher-level IO controller to the IO device
Comment (string)	Text box for a comment on the transfer area.	
LocalToPartnerLength	Specifies the data length of the transfer area that is transferred from the IO device to the higher-level IO controller.	

PartnerToLocalLength	Specifies the data length of the transfer area that is transferred from the higher-level IO controller to the IO device.
LocalAdresses	Specifies the input and output addresses of the transfer area from the local device.
PartnerAdresses	Specifies the input and output addresses of the transfer area in the higher-level IO controller.
TransferUpdateTime(Int32)	Specifies the update time of the transfer area. Only set or queried for a transfer area of the type "TransferAreaType.TM".
PositionNumber	Specifies the number of the virtual submodule of this transfer area.
Type	Specifies the type of transfer area, read-only.
TransferAreaMappingRules	Specifies the routing table of the routing area, read-only.

Deleting transfer areas

To delete transfer areas modify the following program code:

```
Copies the following program code to clipboard.
transferAreaInput.Delete();
```

Iteration through transfer areas

To iterate transfer areas modify the following program code:

```
Copies the following program code to clipboard.
TransferAreaComposition transferAreas = cpItf.TransferAreas;
foreach (TransferArea transferArea in transferAreas)
{
transferArea.Delete();
}
```

The following attributes can be set for IO routing:

Attribute	Description

Offset	Bit-based offset within the routing area to which the data is to be assigned. The length of the offset is determined by the "Begin" and "End" attributes.
Target	Specifies the module or submodule of the IO device that contains the data to be assigned to the configuration of the IO device, a transfer area of the type "TM".
IoType	The “IoType” attribute can only be changed in a transfer area of the “Input” type. In addition, a mixed module must be configured as “Target” for this transfer area. Only then can you select whether the data of the inputs (IoType.Input) are to be read or whether the data of the outputs (IoType.Output) are to be read (back).
Begin	Specifies the beginning of the data to be read by the "Target" attribute.
End	Specifies the end of the data to be read by the "Target" attribute.

Configuration of IO routing

Creating IO routes

To create IO routes modify the following program code:

```
Copies the following program code to clipboard.
//Create TransferAreaMappingRule
TransferAreaMappingRuleComposition routingTable
= transferArea.TransferAreaMappingRules;///
//Create a new IO route
TransferAreaMappingRule route1 = routingTable.Create();
```

Setting attributes of IO routes

The following attributes can be set for IO routing:

Attribute	Description
Offset	Bit-based offset within the routing area to which the data is to be assigned. The length of the offset is determined by the "Begin" and "End" attributes.
Target	Specifies the module or submodule of the IO device that contains the data to be assigned to the configuration of the IO device, a transfer area of the type "TM".

IoType	The “IoType” attribute can only be changed in a transfer area of the “Input” type. In addition, a mixed module must be configured as “Target” for this transfer area. Only then can you select whether the data of the inputs (IoType.Input) are to be read or whether the data of the outputs (IoType.Output) are to be read (back).
Begin	Specifies the beginning of the data to be read by the "Target" attribute.
End	Specifies the end of the data to be read by the "Target" attribute.

Deleting IO routes

To delete IO routes modify the following program code:

```
Copies the following program code to clipboard.

transferAreaMappingRule.Delete();
```

Iteration via IO routing

To iterate via IO routing modify the following program code:

```
Copies the following program code to clipboard.

TransferAreaMappingRuleComposition routingTable = transferArea.TransferA
foreach (TransferAreaMappingRule route in routingTable)
{
route.Delete();
}
```

2.36 Openness transfer areas for PnPn coupler

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project
- To compile the project, all device must be "offline"

Introduction

You can use the TIA Portal Openness to add module and submodule, delete and search transfer areas for PnPn coupler.

Configuration of transfer areas

Creating transfer areas

For example, to create module on the next free position, use the following program code:

```
Copies the following program code to clipboard.

NetworkInterface coupler_Itf =
deviceItem.GetService<NetworkInterface>();

TransferAreaComposition transferAreas =
coupler_Itf.TransferAreas;

// Create a transfer area of type input at next free
positionnumber

TransferArea transferAreaInput =
transferAreas.Create("Input", TransferAreaType.IN);
```

Note

It is not possible to create a submodule without Positionnumber.

For example, to create module on PositionNumber and submodule at the next free submodulepositionnumber at positionNumber, use the following program code:

```
Copies the following program code to clipboard.

// Create a transfer area of type input at positionnumber 5

TransferArea transferAreaInput =
transferAreas.Create("Input", TransferAreaType.IN, 5);
```

The following attributes are used for creating transfer areas:

Attribute	Description

Type	Specifies the type of the transfer area to be created. The following types are possible:	
	TransferAreaType.None	Default value
	TransferAreaType.IN	Transfer area Input
	TransferAreaType.OUT	Transfer area Output
	TransferAreaType.MSI	Transfer area Shared Input (MSI)
	TransferAreaType.MSO	Transfer area Shared Output (MSO)
	TransferAreaType.MSO_LOCAL	Transfer area Local Shared Output (MSO_LOCAL)
	TransferAreaType.RECORD_WRITE_STO	Transfer area for Record Data Write up to 8 data records will be buffered
	TransferAreaType.RECORD_WRITE_PUB	Transfer area for Record Data Write overwrites previous data record
	TransferAreaType.RECORD_READ_STO	Transfer area for Record Data Read reads the oldest data record in the buffer
	TransferAreaType.RECORD_READ_PUB	Transfer area for Record Data Read reads data record written last
	TransferAreaType.MSI_MSO	Transfer area Shared Input (MSI) / Transfer area Shared Output (MSO)
	TransferAreaType.IN_OUT	Transfer area Input / Transfer area Output
	TransferAreaType.LOCAL_RECORD_STO	Transfer area for local data record coupling, up to 8 data records are buffered
	TransferAreaType.LOCAL_RECORD_PUB	Transfer area for local data record coupling, overwrites previous data record

TransferAreaType.None	Default value
TransferAreaType.IN	Transfer area Input
TransferAreaType.OUT	Transfer area Output
TransferAreaType.MSI	Transfer area Shared Input (MSI)
TransferAreaType.MSO	Transfer area Shared Output (MSO)
TransferAreaType.MSO_LOCAL	Transfer area Local Shared Output (MSO_LOCAL)
TransferAreaType.RECORD_WRITE_STO	Transfer area for Record Data Write up to 8 data records will be buffered
TransferAreaType.RECORD_WRITE_PUB	Transfer area for Record Data Write overwrites previous data record
TransferAreaType.RECORD_READ_STO	Transfer area for Record Data Read reads the oldest data record in the buffer
TransferAreaType.RECORD_READ_PUB	Transfer area for Record Data Read reads data record written last
TransferAreaType.MSI_MSO	Transfer area Shared Input (MSI) / Transfer area Shared Output (MSO)
TransferAreaType.IN_OUT	Transfer area Input / Transfer area Output
TransferAreaType.LOCAL_RECORD_STO	Transfer area for local data record coupling, up to 8 data records are buffered
TransferAreaType.LOCAL_RECORD_PUB	Transfer area for local data record coupling, overwrites previous data record
TransferAreaType.SUB_MSI	Transfer area copy of Shared Input (MSI)

TransferAreaType.SUB_MSO	Transfer area copy of Shared Output (MSO)
TransferAreaType.SUB_LOCAL_RECORD_STO_READ	Transfer area for Record Data Read for reading the oldest data record in the buffer
TransferAreaType.SUB_LOCAL_RECORD_PUB_READ	Transfer area for Record Data Read for reading the most recently written data record
TransferAreaType.PROFISAFE_IN12_OUT6	Transfer area with 12 byte input and 6 byte output
TransferAreaType.PROFISAFE_IN6_OUT12	Transfer area with 6 byte input and 12 byte output
TransferAreaType.ISOC_STATUS_CONTROL	Transfer area for isochronous control / status module
TransferAreaType.ISOCHRON_OUT	Transfer area for isochronous output submodules
TransferAreaType.ISOCHRON_IN	Transfer area for isochronous input submodules

Setting attributes of the transfer areas

To set attributes of a transfer area, use the following program code, for example:



```
// Change input length
CouplerTransf.LocalToPartnerLength = 5;
// Change output length
CouplerTransf.PartnerToLocalLength = 5;
// Change name
CouplerTransf.Name = "Testname";
```

Some attributes must be set or queried, but all of them can be set or queried using the general calls TransferArea.GetAttribute() or SetAttribute(). Use the following program code, for example



```
// Set Comment of Transferarea
```

```
string myIndividualComment = "MyIndividualComment";
CouplerTransf.SetAttribute("Comment", myIndividualComment);
// Get positionNumber of Transferarea
Int32 positionNumber =
(Int32)CouplerTransf.GetAttribute("PositionNumber");
```

The following attributes can be set for transfer areas:

Attribute	Description	
Name (string)	Specifies the name of the transfer area	
Direction	Specifies the direction in which the data of the transfer area is transferred. The following directions are possible:	
	TransferAreaDirection.LocalTo-Partner	Data of the transfer area is transferred from the IO device to the higher-level IO controller.
	TransferAreaDirection.partnerTo-Local	Data of the transfer area is transferred from the higherlevel IO controller to the IO device.
	TransferAreaDirection.bidirectional	Data of the transfer area can be transferred in both directions between the higher-level IO controller and the IO device. The "LocalToPartnerLength" attribute determines the length of the transferred data from IO device to the higher-level IO controller. The "PartnerToLocal- Length" attribute determines the amount of data from the higher-level IO controller to the IO device

Comment (string)	Text box for a comment on the transfer area
LocalToPartnerLength	Specifies the input data length of the transfer area
PartnerToLocalLength	Specifies the output data length of the transfer area
PositionNumber	Specifies the slot number of the transfer area It is not possible to create sub module without a position number
ExtendedPositionNumber	Specifies the sub slot number of the transfer area The ExtendedPositionNumber is only required for TransferAreas with more than one submodule like MSI
Type	Specifies the type of transfer area
SharedDeviceAccessConfigured	Controls the access to PLC
UpdateAlarm	Enable the update alarm of the transfer area
RecordIndex	Set the data record number that you will be using when writing the data record

Deleting transfer areas

To delete transfer areas, use the following program code:

```
Copies the following program code to clipboard.

TransferArea TransferAreaInput =
transferAreas.Create("Input", TransferAreaType.IN);

TransferAreaInput.Delete();
```

Searching transfer areas

To get the TransferArea with posotionNumber and extendedPositionNumber, use the following program code:

```
Copies the following program code to clipboard.

// Find a transfer area with Positionnumber and
ExtendedPositionNumber

TransferArea transferAreaFind = transferAreas.Find(4, 3);
```

Note

ExtendedPositionNumber is only needed for TransferAreas with more than one Submodule like MSI

To get the first TransferArea with this positionNumber, use the following program code:



```
// Find a transfer area with Positionnumber
TransferArea transferAreaFind = transferAreas.Find(5);
```

Note

If one of this attributes is not available, you will encounter an exception.

See also

Opening a project

2.37 Openness virtual modules/submodules for ET 200SP PN HF

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to add and delete virtual module/submodule for ET 200 SP PN HF.

The following property are supported in virtual modules:

Property name	Data type	EomAtomValue	EomAtom description
SharedDeviceAccessConfigured	Boolean		
Name	String		
PositionNumber	Int32		
VirtualType	UInt64	1	MsoLocal

AddSubModules	Int64		

The following property are supported in virtual submodules:

Property	Data type	EomAtom value	EomAtom Description
SharedDeviceAccessConfigured	Boolean		
Comment	String		
PositionNumber	Int32		
Name	String		
ActivateDataStatus	Boolean		
InputAddressLength	Int64		
OutputAddressLength	Int64		
VirtualSubType	UInt64	1	MsoLocal
		2	MsoSub

Program code: Create and delete virtual module

Modify the following program code to add a new module:

```
Copies the following program code to clipboard.
string Type = "OrderNumber:6ES7 155-6AU30-0CN0/V4.2";
Device ET200SP = newProject.Devices.CreateWithItem(Type,
"ET200SP", "ET200SP");
DeviceItem Rack = ET200SP.DeviceItems.First();
string TypeIdentifier = "OrderNumber:ET200SP.Virtual.Module";
string Name = "VirtualIO_1";
int PositionNumber = 100;
DeviceItem VIM = Rack.PlugNew(TypeIdentifier, Name,
PositionNumber);
```

Modify the following program delete a module:

```
Copies the following program code to clipboard.
DeviceItem Rack = ET200SP.DeviceItems.First();
string TypeIdentifier = "OrderNumber:ET200SP.Virtual.Module";
string Name = "VirtualIO_1";
int PositionNumber = 100;
DeviceItem VIM = Rack.PlugNew(TypeIdentifier, Name,
PositionNumber);
VIM.Delete();
```

Program code: Create and delete virtual sub module

Modify the following program code to create virtual submodules in Openness:

```
Copies the following program code to clipboard.
VIM1.SetAttribute("AddSubModules", (Int64)1);
VIM2.SetAttribute("AddSubModules", (Int64)2);
```

Note

To add a submodule, you have to set the module property "AddSubModules" to count of added submodules. If you try to add more submodulesthan allowed, there is an exception.

Modify the following program code to delete a submodule:

```
Copies the following program code to clipboard.
SubModule3.Delete();
```

Note

You cannot delete the first two submodules.

2.38 Accessing domain settings

Requirements

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal

- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to access the complete domain management. The following two new services are added to the Subnet class for accessing domain management via TIA Portal Openness:

- service MrpDomainOwner
- service SyncDomainOwner

Program code: Accessing MrpDomainOwner

The service MrpDomain provides the navigator MrpDomains which contains the MrpDomainComposition. The composition contains objects of type MrpDomain.



```
Copies the following program code to clipboard.

Subnet subnet = …;

MrpDomainOwner mrpDomainOwner = subnet.GetService<MrpDomainOwner>
();

MrpDomainComposition mrpDomainComposition =
mrpDomainOwner.MrpDomains;
```

Modify the following program code to create a new domain with a name newMrpDomain;



```
Copies the following program code to clipboard.

int countOfMrpDomains = mrpDomainComposition.Count;

MrpDomain someMrpDomain = mrpDomainComposition.ElementAt(3);

MrpDomain firstMrpDomain = mrpDomainComposition.First();

MrpDomain byName = mrpDomainComposition.Find("DomainName");

// create a new domain

MrpDomain newMrpDomain =
mrpDomainOwner.MrpDomains.Create("newMrpDomain");
```

Modify the following program code to read/write attributes value of a MRP domain objects:



```
Copies the following program code to clipboard.

NetworkInterface toBeAdded = …;

newMrpDomain.SetAttribute("IsDefault", true);

newMrpDomain.SetAttribute("ManagerOutsideOfProjectActive", false);
```

```
var participants = firstMrpDomain.DomainParticipants;
int count = participants.Count;
participants.Add(toBeAdded);
foreach (NetworkInterface networkIf in participants)
{
// do something at the interface
}
newMrpDomain.Delete();
```

Modify the following program code to create multiple MRP instances:



```
var mrpInstancesService =
interfaceSubmodule.GetService<MrpInstancesOwner>();
//MrpInstancesOwner service provides the composition of
MrpInstances.
var mrpInstances = mrpInstancesService.MrpInstances;
//Individual MRP Instances can be accessed in the
composition:
var mrpInstance = mrpInstances[0];
//Properties of an MRP instance can be read, for example:
var value = mrpInstance.ConnectedMrpDomain;
//Dynamic attributes of MRP instance can accessed:
var value2 = mrpInstance.GetAttribute("MediaRedundancyRole");
//Attributes can be set:
mrpInstance.SetAttribute("MediaRedundancyRole",
MediaRedundancyRole.Manager);
```

Program code: Accessing SyncDomainOwner

The service provides the navigator SyncDomains which contains the SyncDomainComposition. This composition contains objects of type SyncDomain.



```
Subnet subnet = …;
```

```
SyncDomainOwner syncDomainOwner =
subnet.GetService<SyncDomainOwner>();
SyncDomainComposition syncDomainComposition =
syncDomainOwner.SyncDomains;
```

Modify the following program code to create a new SyncDomain with specific name:

```
 Copies the following program code to clipboard.
int countOfSyncDomains = syncDomainComposition.Count;
SyncDomain someSyncDomain =
syncDomainComposition.ElementAt(3);
SyncDomain firstSyncDomain = syncDomainComposition.First();
SyncDomain byName = syncDomainComposition.Find("DomainName");
// create a new domain
SyncDomain newSyncDomain =
syncDomainOwner.SyncDomains.Create("newSyncDomain");
```

Modify the following program code to read/write the attribute value of Sync Domain:

```
 Copies the following program code to clipboard.
NetworkInterface toBeAdded = …;
string convertedName = newSyncDomain.ConvertedName;
newSyncDomain.SetAttribute("IsDefault", true);
newSyncDomain.SetAttribute("HighPerformanceActive", true);
newSyncDomain.SetAttribute("FastForwardingActive", true);
var participants = firstSyncDomain.DomainParticipants;
int count = participants.Count;
participants.Add(toBeAdded);
foreach (NetworkInterface networkIf in participants)
{
// do something at the interface
}
newSyncDomain.Delete();
```

See also

Connecting to the TIA Portal
Opening a project

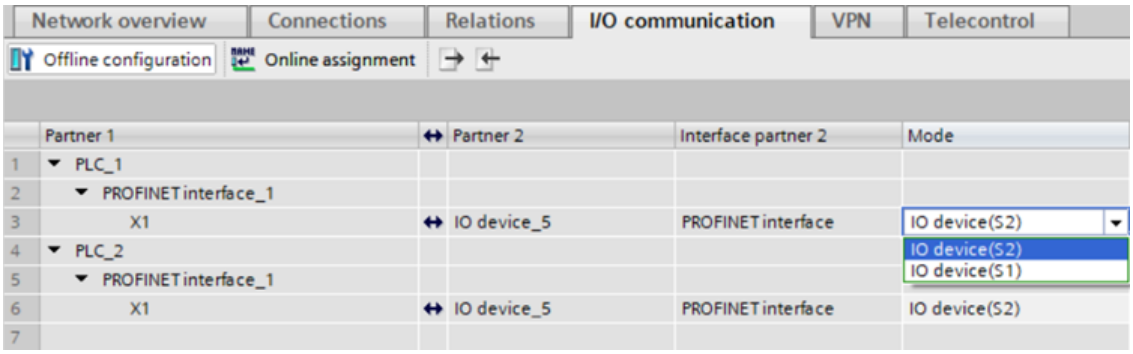
2.39 Accessing redundancy mode attribute

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a Project

Application

You can use the TIA Portal Openness to access an S2-capable device to be operated as a Switched S1 device. Using Openness, you should be able to change the Mode setting on the TIA Portal user interface for an IOD that is capable of system redundancy of type S2 to 'S1' as shown below:



The Mode property of TIA Portal is available in the TIA Portal Openness with name RedundancyMode. This attribute is available on S2 and R1 IO devices' Profinet interface.

The usage of this attribute is shown in the following table.

Attribute	Description
NetworkInterface.GetAttribute("RedundancyMode")	Reading the attribute via Openness
NetworkInterface.SetAttribute("RedundancyMode", "RedunancyMode.S2")	Setting the attribute via Openness
NetworkInterface.SetAttribute("RedundancyMode", "RedunancyMode.SwitchedS1")	

Program code

Modify the following program code to access redunancy mode attribute:

```
Copies the following program code to clipboard.
```

```
Device S2device = ...;
string s_S2DeviceInterface1Name = "";
NetworkInterface networkInterface1 =
S2device.DeviceItems.First(f => f.Name ==
s_S2DeviceInterface1Name).GetService<NetworkInterface>();
string s_RedundancyMode = "...";
networkInterface1.SetAttribute(s_RedundancyMode,
RedundancyMode.SwitchedS1);
//Assert
Assert.AreEqual(RedundancyMode.SwitchedS1,
networkInterface1.GetAttribute(s_RedundancyMode);
//Act
var systemBlockGroups = plc1?.GetService<ICompilable>();
CompilerResult compilerResult = systemBlockGroups?.Compile();
//Assert
Assert.AreEqual(0, compileResult?.ErrorCount);
```

See also

Connecting to the TIA Portal
Opening a project

2.40 Creating CCDX transfer area

Requirement

- The application is connected to the TIA Portal via TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the Create() of the MulticastableTransferArea object in the TIA Portal Openness to create a CCDX transfer area. The Multicastable refers to a one to many relation that means a sender transfer area can cast information to multiple receiver partner transfer areas. Whereas in the opposite direction a receiver transfer area can communicate with only one sender partner.

Method

The overloading Create() accepts the following parameter name in Openness:

Parameter	Return type	Description
Create(NetworkInterface, TransferAreaType)	MulticastableTransferArea	Creates a Transfer Area and adds it to an existing CCDX connection with the specified type (e.g. DDX) on the specified network interface
Create(NetworkInterface, TransferAreaType, Name)	MulticastableTransferArea	Creates a Transfer Area by the sepcified name and adds it to an existing CCDX connection with the specified type (e.g. DDX) on the specified network interface
Create(NetworkInterface, TransferAreaType, Name, Length)	MulticastableTransferArea	Creates a Transfer Area by the sepcified name and length and adds it to an existing CCDX connection with the specified type (e.g. DDX) on the specified network interface.
Create(MulticastableTransferArea, TransferAreaType)	MulticastableTransferArea	Creates a new Transfer Area with the specified type and assigns it as receiver to an existing sender side transfer area.

Program code

```
Copies the following program code to clipboard.
// Given the NetworkInterfaces
private void CreatingTransferArea()
{
```

```
NetworkInterface senderNetworkInterface = ...;
NetworkInterface receiverNetworkInterface = ...;
MulticastableTransferArea transferArea1 =
senderNetworkInterface.MulticastableTransferAreas.Create(receiverNetwor
TransferAreaType.DDX);
string name = "Example_CCDX_TA";
MulticastableTransferArea transferArea2 =
senderNetworkInterface.MulticastableTransferAreas.Create(receiverNetwor
TransferAreaType.DDX, name);
int length = 32;
MulticastableTransferArea transferArea3 =
senderNetworkInterface.MulticastableTransferAreas.Create(receiverNetwor
TransferAreaType.DDX, name, length);
// Create receiver only
MulticastableTransferArea transferArea = ...;
MulticastableTransferArea transferArea4 =
senderNetworkInterface.MulticastableTransferAreas.Create(transferArea,
TransferAreaType.DDX);
}
```

See also

Opening a project

2.41 Deleting CCDX transfer area

Requirement

- The application is connected to the TIA Portal via TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the Delete() of the MulticastableTransferArea object in the TIA Portal Openness to delete the transfer area. The Multicastable refers to a one to many relation that means a sender transfer area can cast information to multiple receiver partner transfer areas. Whereas in the opposite direction a receiver transfer area can communicate with only one sender partner.

The Delete() behaves differently when deleting a receiver. There is always only 1 sender (One to Many).

Sender	Receiver	Behaviour
1	c = 1	Both the sender and the receiver will be deleted
1	c>1	Only the receiver will be deleted

The following behavior expected when deleting the sender:

Sender	Receiver	Behaviour
1	Any	Both the sender and all receivers will be deleted

Program code

```
Copies the following program code to clipboard.
private void DeleteTransferArea()
{
// Given the selected NetworkInterface
NetworkInterface networkInterface = ...;
// The name of the TransferArea
string transferAreaName = "Example_CCDX_TA";
networkInterface.MulticastableTransferAreas.FirstOrDefault(item
=> item.Name == transferAreaName).Delete();
}
```

See also

Opening a project

2.42 Modifying CCDX transfer areas configuration

Requirement

- The TIA Portal is connected to the TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to modify the configuration of CCDX transfer areas. In Openness, you can access transfer area via NetworkInterface as TransferAreaMulticastableComposition objects which contain all the transfer areas that belong to the same interface of the same PLC .

The Multicastable refers to a one to many relation that means a sender transfer area can cast information to multiple receiver partner transfer areas. Whereas in the opposite direction a receiver transfer area can communicate with only one sender partner. (eg. CCDX transfer area).

Property

Property name	Data type	Description	Access
Address	AddressComposition	Gets the address information such as Start address, Length, IoType, etc. of a transfer area of type Siemens.Engineering.HW.Address	Read/Write
Parent	IEngineeringObject	Gets the EOM parent of a transfer area	Read-only
Comment	String	Gets the comment property of a transfer area	Read/Write
Direction	TransferAreaDirection	Gets the direction property of a transfer area as of type TransferAreaDirection. This indicates the direction of the communication depends on whether the transfer area is a sender or a receiver.	Read-only
DataLength	Int	Gets/sets the Data length (i.e. address length) of a transfer area	Read/Write

Name	String	Gets the name property of a transfer area	Read/Write
PartnerTransferAreas	MulticastableTransferAreaAssociation	Gets the partner transfer areas of a multicastable transfer area for eg. CCDX For example, sender transfer area contains its receiver transfer area partner(s) on the partner device (s). A sender transfer area can have multiple receiver partners. Both a sender or a receiver transfer area must have at least one partner, they cannot exist alone. Therefore, when deleting a sender transfer area, all it's receiver partners have to be	

		deleted as well. But deleting a receiver transfer area doesn't necessarily come with the deletion of its sender partner if it has additional receiver partner.	Read only
Type	TransferAreaType	Gets the Type property of a transfer area as of type TransferAreaType A multicastable transfer area can have below enum value: • TransferAreaType.DDX where DDX refers to transfer area type 'CCDX' for Direct Data Exchange communication.	Read only
MulticastableTransferAreas	MulticastableTransferAreaComposition	Gets the MulticastableTransferAreas property of the networkinterface. It contains multicastable transfer areas (for eg. CCDX transfer areas) of a network interface. These can be both sender or receiver side transfer areas.	Read only

Attribute	Data type	Description	Access
TransferUpdateTime	Int32	Update time for the transmission of the provider (sender) item	Read-only

Program code

```
Copies the following program code to clipboard.

private void ModifyingCCDXTransferArea()
```

```
{
// Given the selected NetworkInterface
NetworkInterface networkInterface = ...;
// Navigate to the desired MulticastableTransferArea
string transferAreaName = "Example_CCDX_TA";
MulticastableTransferArea transferArea =
networkInterface.MulticastableTransferAreas.FirstOrDefault(item
=> item.Name ==
transferAreaName);
// Properties
Propertis properties = transferArea.Properties;
// Address
Address address = transferArea.Address;
// Parent
IEngineeringObject parent = transferArea.Parent;
// Comment
string comment = transferArea.Comment;
// Direction
TransferAreaDirection direction = transferArea.Direction;
// Name
string name = transferArea.Name;
// PartnerTransferAreas
MulticastableTransferAreaAssociation partnerTransferAreas =
transferArea.PartnerTransferAreas;
// TransferUpdateTime
int transferUpdateTime =
(int)transferArea.GetAttribute("TransferUpdateTime");
// Type
TransferAreaType type = transferArea.Type;
// DataLength
transferArea.DataLength = 12;
}
```